

Milestone 2 Report

SCIPER 392412 · 373176 · 398388

1 Introduction

We are back with the Gaming and Mental Health dataset! Quick recap: we have 2,000 samples total, keeping 1,600 for training and 400 for testing.

Our two tasks remain the same. First, we predict the continuous addiction score (0 to 10). Second, we classify players into Low, Medium, or High addiction levels. For this milestone, we ditch the simple models from Milestone 1 and build a Multi-Layer Perceptron (MLP) and a K-Means clustering algorithm from scratch.

2 Methods

We prepared the data by normalising it with the mean and standard deviation of our training set. To avoid cheating (data leakage!), we carved out an 80/20 train/validation split from our 1,600 training samples. This left us with 1,280 samples for training and 320 for validation.

2.1 Multi-Layer Perceptron (MLP)

Our neural net is written entirely from scratch. Weights are initialized using a normal distribution scaled by $1/\sqrt{I}$ (where I is the input dimension), and biases start at zero.

The forward pass runs layer-by-layer: $\mathbf{A}^{(l)} = \mathbf{Z}^{(l-1)}\mathbf{W}^{(l)\top} + \mathbf{b}^{(l)}$ followed by the activation $\mathbf{Z}^{(l)} = \sigma^{(l)}(\mathbf{A}^{(l)})$, where $\mathbf{Z}^{(0)}$ is the input. We coded both ReLU and Sigmoid activations.

For training, backpropagation propagates errors backwards using the chain rule. We update parameters using Mini-Batch Gradient Descent (batch size of 32). We also shuffle data indexes before each epoch so the network doesn't get stuck in repetitive loops.

We used two loss metrics:

- **Mean Squared Error (MSE):** For regression.
- **Categorical Cross-Entropy:** For classification. We added a tiny offset ($\epsilon = 10^{-9}$) to avoid log-zero errors.

2.2 K-Means Clustering

K-Means is unsupervised, so we had to get creative to use it for classification. The algorithm groups our 13 features into K clusters. We initialize centroids by picking random data points. We assign points to the nearest centroid using Euclidean distance: $d(\mathbf{x}, \mathbf{c}) = \|\mathbf{x} - \mathbf{c}\|_2$. Centroids are recalculated as the mean of their assigned points. If a cluster goes empty, we jumpstart it by picking a new random point.

To classify, we assign each cluster a label based on the majority vote of the true labels of training points inside it. For predictions, test points just take the label of their nearest cluster center. It is surprisingly simple and works!

3 Experiments and Results

3.1 K-Means Clustering

We ran a parameter sweep for K values from 1 to 20 to see how cluster count affects performance.

The best accuracy was at $K = 5$, getting 73.12% validation accuracy, though Macro F1 was quite poor (0.4172). However, when we pushed K to 18, the Macro F1 shot up to 0.5943, even though accuracy dropped slightly to 70.94%. This was a real eye-opener! Having more clusters allows the model to capture minority classes instead of letting them get drowned out in majority votes.

3.2 MLP Tuning (Classification and Regression)

Next up was tuning the MLP (64 hidden units). We ran a grid search for learning rates (lr) and training epochs for both tasks.

For classification (ReLU/Sigmoid activations), the best run hit 76.56% accuracy (F1: 0.4541) at $lr = 0.01$ and 500 epochs (see Fig 1, left). We noticed that tiny learning rates ($lr = 10^{-4}$) converged at a snail's pace. On the flip side, a high learning rate ($lr = 0.5$) made the gradients go haywire, resulting in worse accuracy.

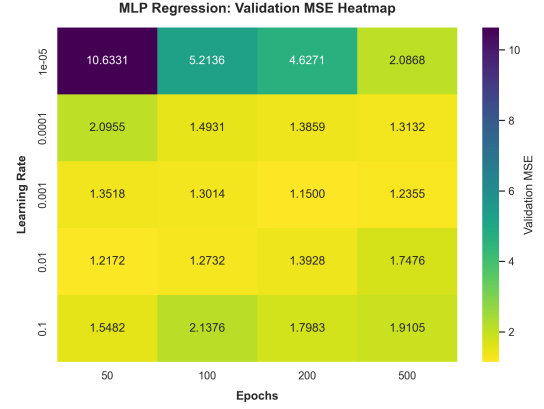
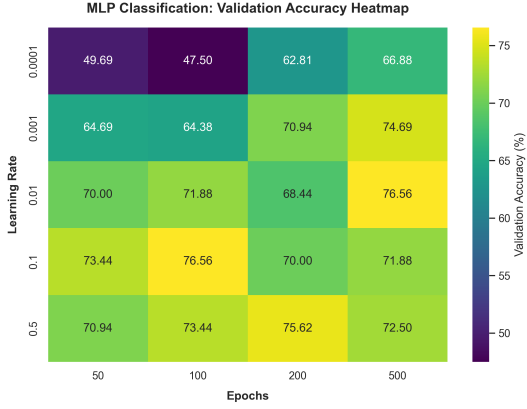


Figure 1: MLP validation performance: Classification Accuracy (left) and Regression MSE (right)

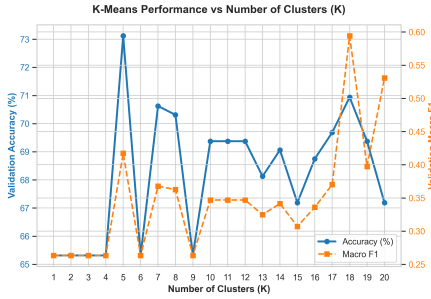


Figure 2: K-Means validation scores vs. K

For regression (ReLU/ReLU activations), our default settings ($lr = 10^{-5}$, 100 epochs) got a validation MSE of 5.2136. After sweeping the parameters, we found a sweet spot at $lr = 0.001$ and 200 epochs. This dropped our validation MSE to a fantastic 1.1500! (see Fig 1, right). Going higher than $lr = 0.01$ caused the model to overfit or diverge.

3.3 Summary of Results

Table 1 compares our best configurations for each approach.

Task	Model (Parameters)	Performance
Classification	K-Means ($K = 5$)	Acc: 73.12%, F1: 0.4172
Classification	K-Means ($K = 18$)	Acc: 70.94%, F1: 0.5943
Classification	MLP ($lr = 0.01$, ep=500)	Acc: 76.56%, F1: 0.4541
Regression	MLP ($lr = 0.001$, ep=200)	MSE: 1.1500

Table 1: Summary of final results

4 Extensions

To make our pipeline more robust, we implemented several advanced techniques:

- **K-Means++ Initialization:** Instead of pure random selection, we implemented K-Means++ and choosing initial centroids proportional to their squared distance to the nearest existing center. This helps standard K-Means avoid poor local optima.
- **Categorical Cross-Entropy:** We expanded binary cross-entropy to handle multi-class labels natively, adding a stability buffer ($\epsilon = 10^{-9}$) to avoid log-zero errors.
- **Deep MLP Architectures:** We tried a two-hidden-layer network ($D, 64, 32, C$). Surprisingly, it did not help! Validation accuracy dropped slightly, probably because the extra parameters overfit our small dataset.
- **Random Batch Shuffling:** We shuffled data indexes at each epoch. This randomizes batch composition and prevents cyclic gradient paths.

5 Discussion and Conclusion

Coding these methods was a fantastic learning experience.

For regression, our tuned MLP reached an MSE of 1.1500. This shows how powerful non-linear neural nets can be when handling complex relationships.

For classification, the supervised MLP outperformed the unsupervised K-Means clustering (76.56% vs 73.12%). This makes sense because the MLP is directly guided by the true labels.

Overall, parametric models are a big step up from Milestone 1 baselines. But hyperparameter tuning is absolutely critical to avoid slow learning or numerical instability!